Lab 7: Fuzzing for Input Validation Bugs

Name: Sakshi P Shetty **SRN:** PES1UG24AM395 **Section:** G

Repository: SETAPESU26/L6_L7_63_house_painting_quote (LAB7 folder)

Module under test: house_painting_quote_parser.py **Tests:** test_house_painting_quote_parser.py

Environment: Windows, Python 3.12.7, pytest 9.1.1, Hypothesis 6.170.15

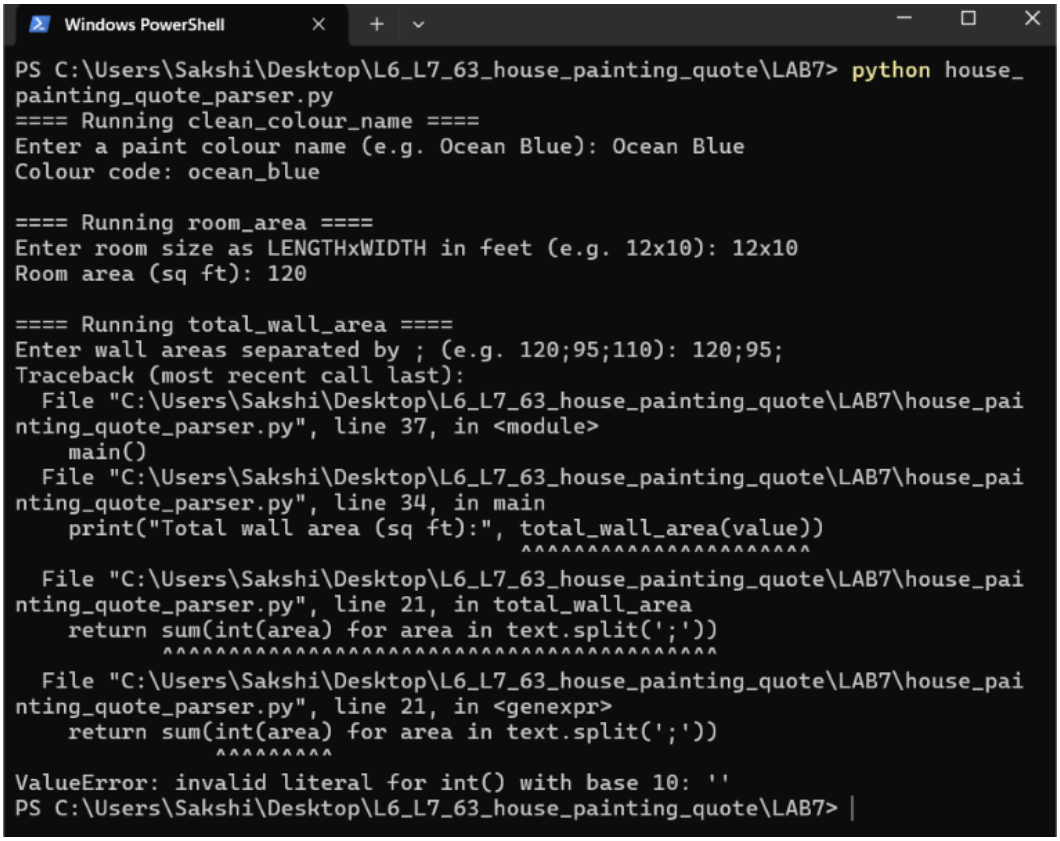
Objective

To use property-based fuzz testing with the Hypothesis library to find input-validation bugs in the house painting quote parser (functions *clean_colour_name*, *room_area* and *total_wall_area*), and then harden the module so that every function handles any input without crashing.

Approach

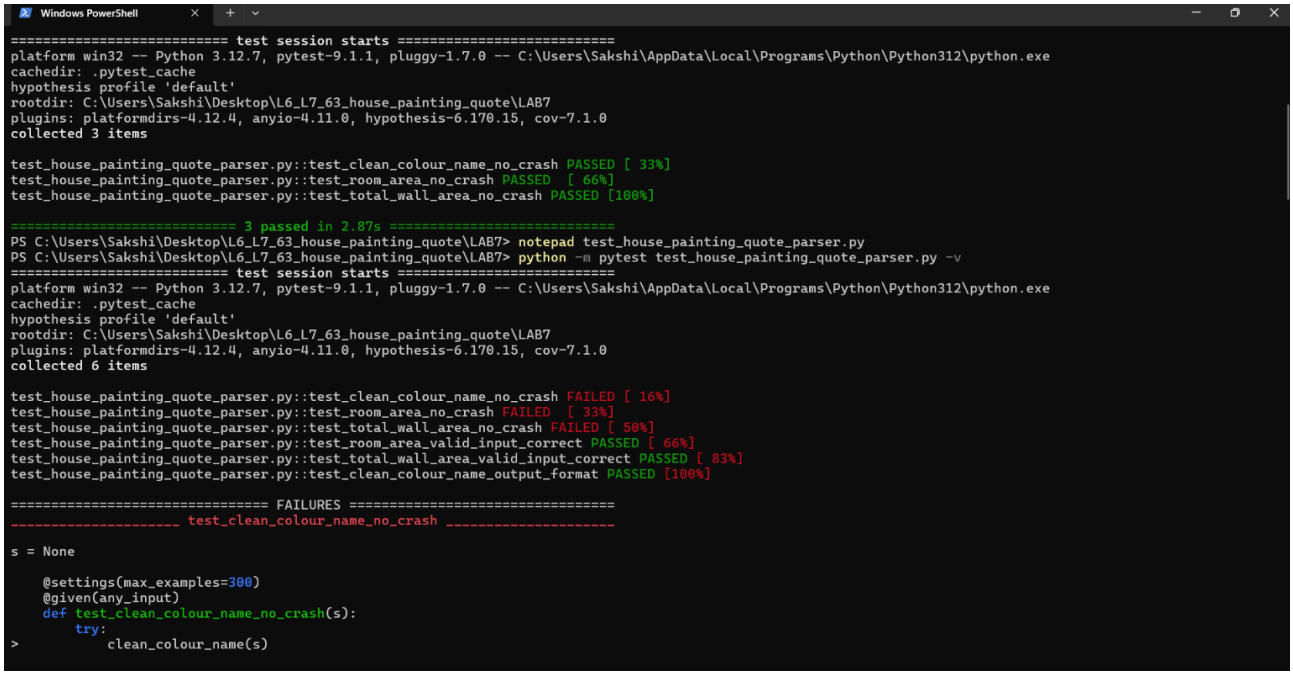
Each function received a "no-crash" property test: Hypothesis feeds it random Unicode text, *None*, and number-like strings built from digits, signs, 'x', ';', a superscript '²' and a non-English digit; any exception counts as a failure. Three extra property tests check that valid input still produces the correct result after the fix (e.g. room_area(f"{L}x{W}") == L*W for random positive L and W).

SS1 - Running the buggy module

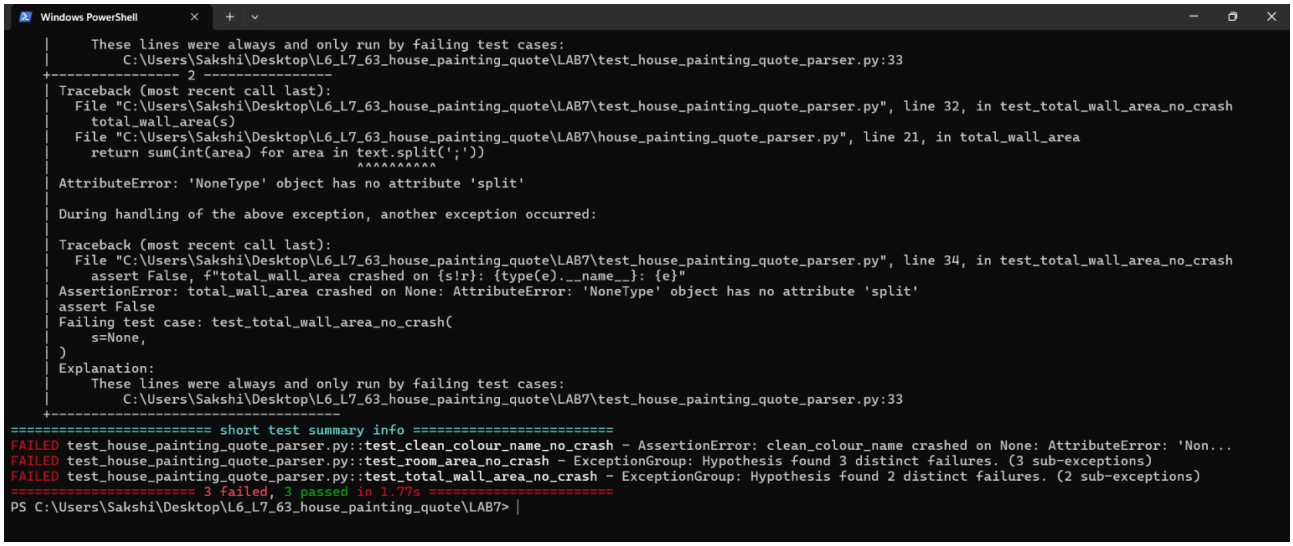


SS1: *python house_painting_quote_parser.py* on the original module. clean_colour_name and room_area work, but total_wall_area crashes on the input '120;95;' with **ValueError: invalid literal for int() with base 10: ''**.

SS2 - Hypothesis tests failing on the buggy module



SS2a: 6 tests collected; the three no-crash fuzz tests FAIL on the buggy module.



SS2b: Summary - **3 failed, 3 passed**. Hypothesis reports 1, 3 and 2 distinct failures respectively.

Bugs found by fuzzing

Function	Minimal failing input (Hypothesis)	Error raised	Root cause
clean_colour_name	None	AttributeError: 'NoneType' object has no attribute 'strip'	No type check
room_area	'X'	ValueError: invalid literal for int() with base 10: ''	Empty side of the 'x' not validated
room_area	''	ValueError: not enough values to unpack (expected 2, got 1)	Assumes exactly one 'x'
room_area	None	AttributeError: 'NoneType' object has no attribute 'lower'	No type check
total_wall_area	'' (also '120;95;')	ValueError: invalid literal for int() with base 10: ''	Empty entries passed to int()
total_wall_area	None	AttributeError: 'NoneType' object has no attribute 'split'	No type check

Silent logic bugs (wrong answer, no crash)

Beyond crashes, the original room_area also accepted inputs that are not valid room sizes, because Python's int() is permissive:

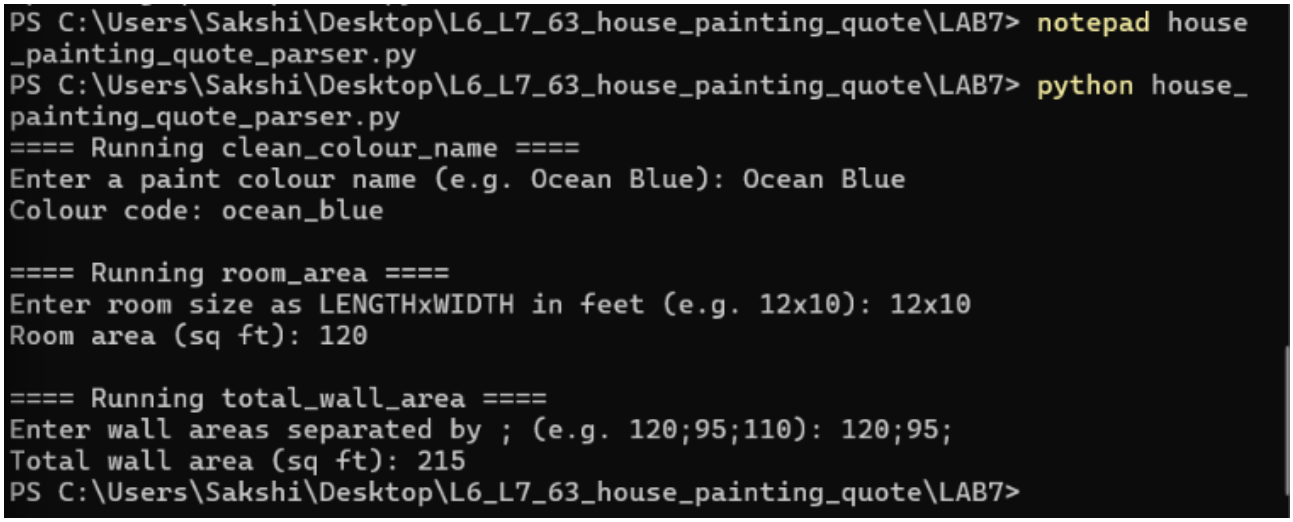
Input	Original result	Fixed result	Why it is wrong
'-5x10'	-50	None	Negative room dimension
'٣x٣' (Arabic-Indic 3)	9	None	int() accepts non-ASCII Unicode digits
'1_0x2'	20	None	int() accepts underscores as digit separators

Fixes applied (defensive programming)

Function	Fix
All three	Type check: non-string input (e.g. None) returns None instead of raising. Inputs longer than 1000 characters are rejected (DoS guard).
Helper _parse_whole_number	Accepts only plain ASCII digits 0-9 (str.isascii() and str.isdigit()), max 9 digits; rejects signs, decimals, underscores, superscripts and non-English digits. This also avoids the int() quirk where '²'.isdigit() is True but int('²') raises.
clean_colour_name	Blank/empty names return None; repeated spaces collapse into a single '_' (' Ocean Blue ' -> 'ocean_blue').
room_area	Requires exactly two parts around 'x'/'X'; both must be positive whole numbers, otherwise returns None.
total_wall_area	Skips empty entries ('120;;95', trailing ';'); returns None if any entry is not a whole number or no valid area exists.
main()	Prints a clear 'Invalid ...' message instead of crashing when a function returns None.

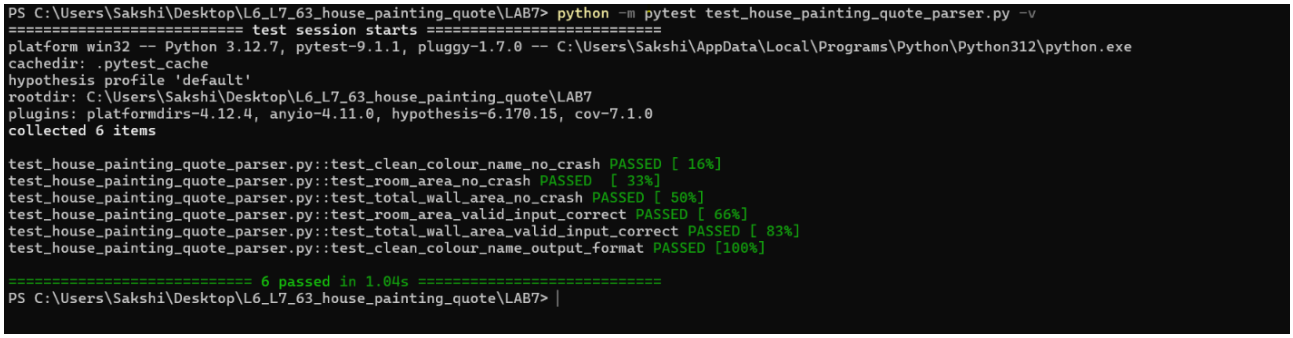
Design choice: invalid input returns *None* rather than raising an exception, because the lab's no-crash tests count any exception as a failure. Specific checks were used instead of a blanket try/except, so real bugs are not hidden.

SS3 - Running the fixed module



SS3: The fixed module with the same inputs - 'ocean_blue', 120 and 215 (the empty entry after the last ';' is skipped). No crash.

SS4 - Hypothesis tests passing on the fixed module



SS4: **6 passed** - all three no-crash fuzz tests and all three correctness property tests pass on the fixed module.

Conclusion

Hypothesis found six crashing inputs across all three functions within seconds, including cases (None, an empty string, a lone 'X') that a hand-written unit test would likely miss. After adding type checks, strict number parsing, empty-entry handling and length limits, the module handles any input without crashing while still returning correct results for valid input.